Lecture 3

Combinational and Sequential Circuits

Computer Organization and Digital Foundations

Dr. Yatish Bathla

PhD.(Information Technology), Obuda University, Budapest

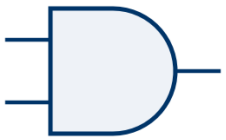
Today's Roadmap

1. Distinguish combinational circuits from sequential circuits
2. Analyse and build half adders, full adders and multi-bit adders
3. Use decoders and multiplexers as building blocks
4. Explain how latches, flip-flops, registers and counters store and move bits

60-minute plan

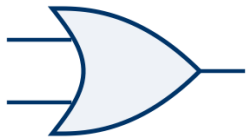
0-12 min	Orientation and recap of gates (slides 1-5)
12-29 min	Combinational building blocks (slides 6-10)
29-52 min	Sequential circuits and state (slides 11-17)
52-60 min	Practice, mistakes, summary (slides 18-20)

Recap: The Logic Gates



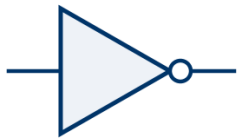
AND

A·B



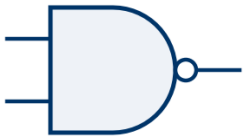
OR

A + B



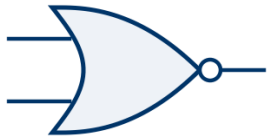
NOT

A′



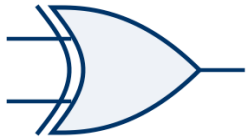
NAND

(A·B)′



NOR

(A+B)′



XOR

A ⊕ B

A	B	AND	OR	NAND	NOR	XOR
0	0	0	0	1	1	**0**
0	1	0	1	1	0	**1**
1	0	0	1	1	0	**1**
1	1	1	1	0	0	**0**

XOR is 1 when the inputs differ: this is the sum bit of an adder.

AND is 1 only when both inputs are 1: this is the carry of an adder.

NAND and NOR are universal: any digital circuit can be built from NAND gates alone.

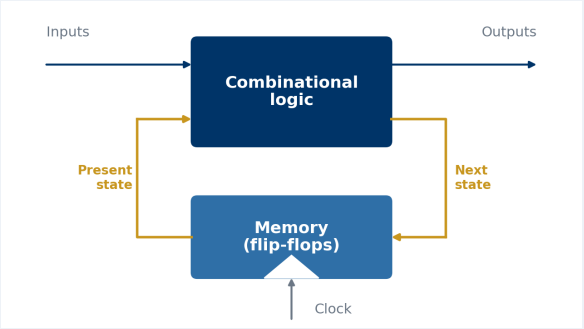
Combinational versus Sequential Circuits

Combinational



- Output depends **only on present inputs**
- No memory, no clock needed
- Examples: adder, decoder, multiplexer

Sequential



- Output depends on **inputs and stored state**
- Has memory (flip-flops) and a clock
- Examples: register, counter, state machine

Designing a Combinational Circuit

1. **State the problem**
2. **List inputs and outputs**
3. **Write the truth table**
4. **Simplify the Boolean function**
5. **Draw the gate circuit**
6. **Verify by simulation**

Worked example: majority of 3

F = 1 when at least two of A, B, C are 1.

Minterms: F = Σm(3, 5, 6, 7)

Simplified: **F = AB + AC + BC**

Cost check

- 3 AND gates + 1 OR gate
- 2 gate levels: short delay
- Verify: try all 8 input rows

The same six steps design every circuit in this lecture.

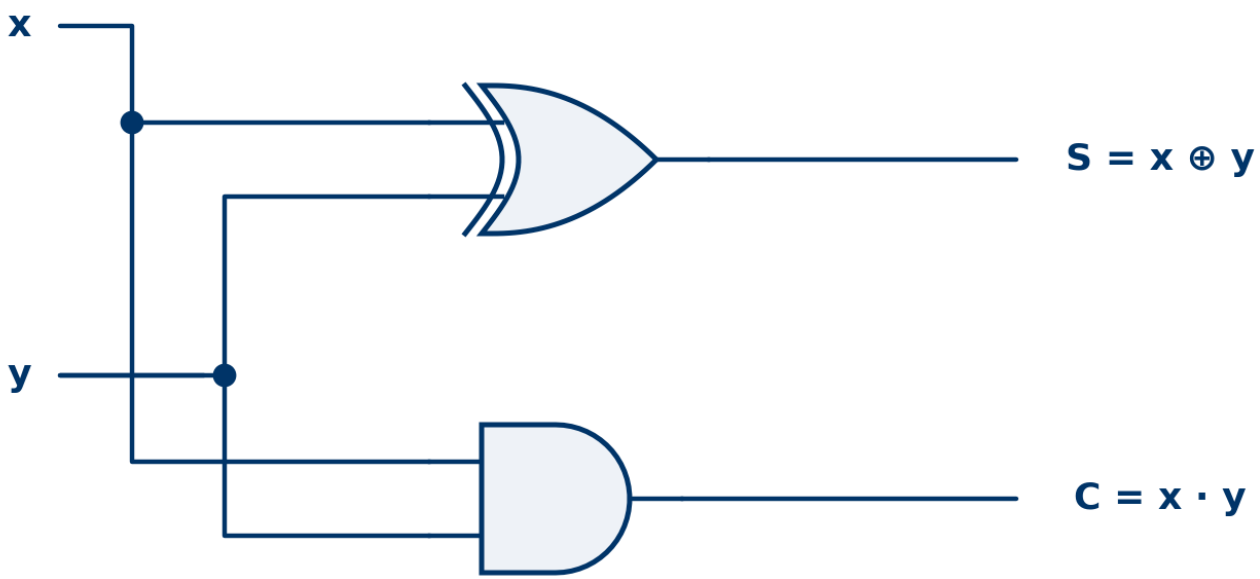
Half Adder: Adding Two Bits

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

S = x ⊕ y (sum)

C = x · y (carry)

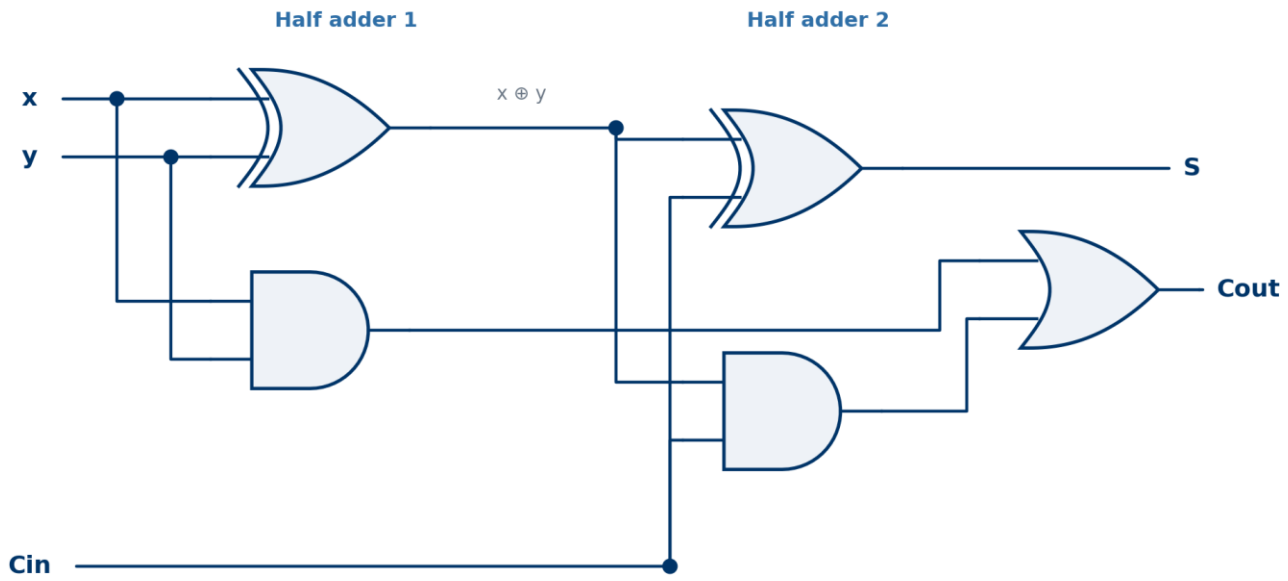
1 + 1 = 10 in binary: sum 0, carry 1



A half adder adds two bits but has no carry input, so it cannot be chained.

Full Adder: Adding Three Bits

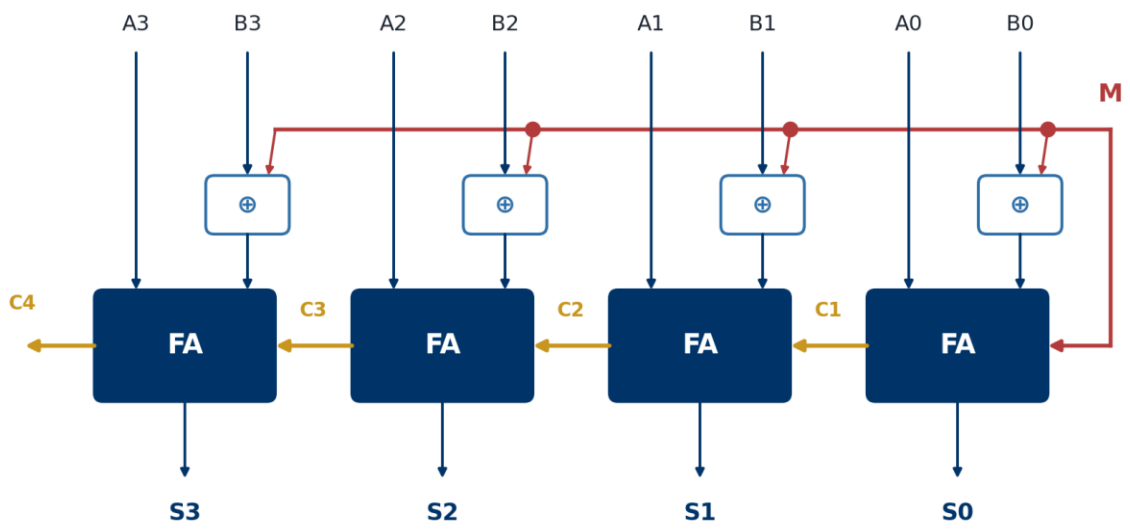
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



S = x ⊕ y ⊕ z **Cout = xy + z(x ⊕ y)** (z is the carry-in)

Full adder = two half adders + one OR gate. Now carries can be chained.

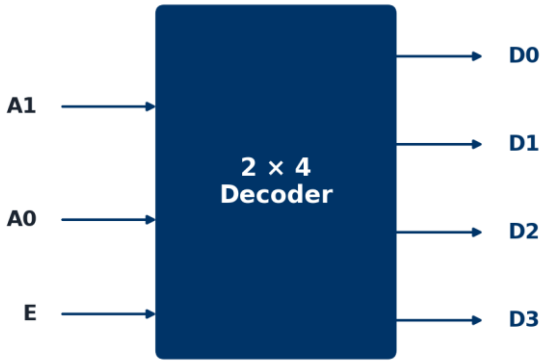
Ripple-Carry Adder and Subtractor



Example	b3	b2	b1	b0
Carries (C3..C0 in)	1	1	1	0
A = 0111 (7)	0	1	1	1
B = 0101 (5)	0	1	0	1
Sum = 1100 (12)	1	1	0	0

- **Chain n full adders:** carry C0 enters the lowest stage and ripples upward.
- **Delay grows with n:** worst case the carry crosses every stage.
- **Subtract with the same hardware:** A − B = A + B' + 1. XOR gates flip B when M = 1; M also feeds C0.
- **Signed overflow** when C4 ⊕ C3 = 1 (here 7 + 5 overflows 4-bit signed).

Decoders and Encoders



E = 1 (enabled)

Exactly one output is 1

E = 0: all outputs 0

A1	A0	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

DEC

Decoder: n inputs → 2^n outputs

Activates exactly one line for each input code.

Used for memory address decoding, chip select, instruction decode.

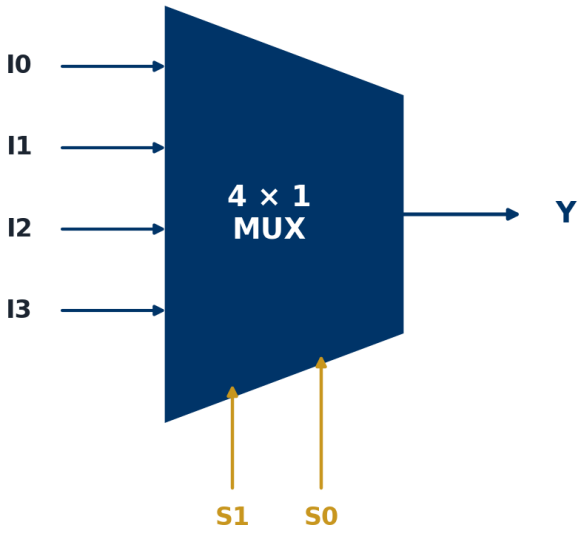
ENC

Encoder: 2^n inputs → n outputs

The reverse operation: reports the number of the active input.

A priority encoder picks the highest request (keyboards, interrupts).

Multiplexers: The Digital Selector



S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Y = S1'S0'I0 + S1'S0 I1 + S1S0' I2 + S1S0 I3

n select lines choose 1 of 2^n inputs.

Function trick

F = A ⊕ B on a 4:1 MUX:

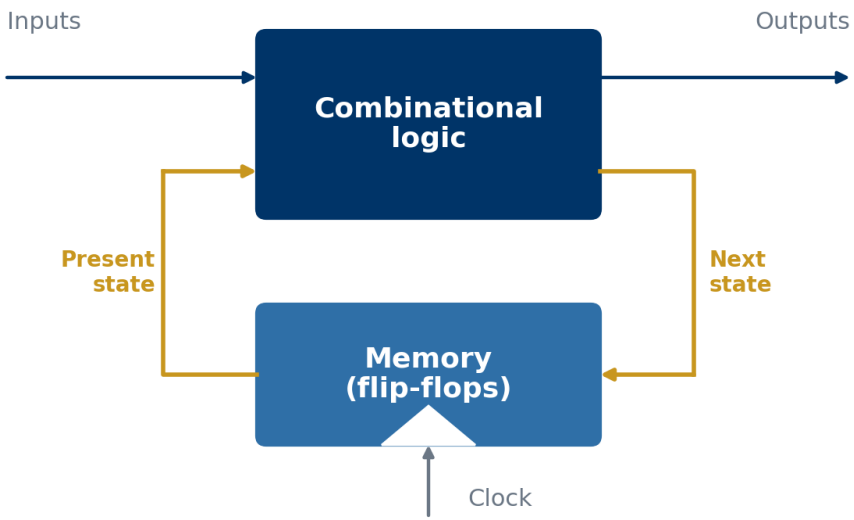
S1 = A, S0 = B

I0 = 0, I1 = 1, I2 = 1, I3 = 0

Any 2-variable function: just tie the data inputs to 0 or 1.

A multiplexer routes one of many inputs to one output: it is how a CPU chooses its data source.

Sequential Circuits: State and the Clock



Synchronous: all changes at clock edges (our focus).
Asynchronous: changes whenever inputs change.

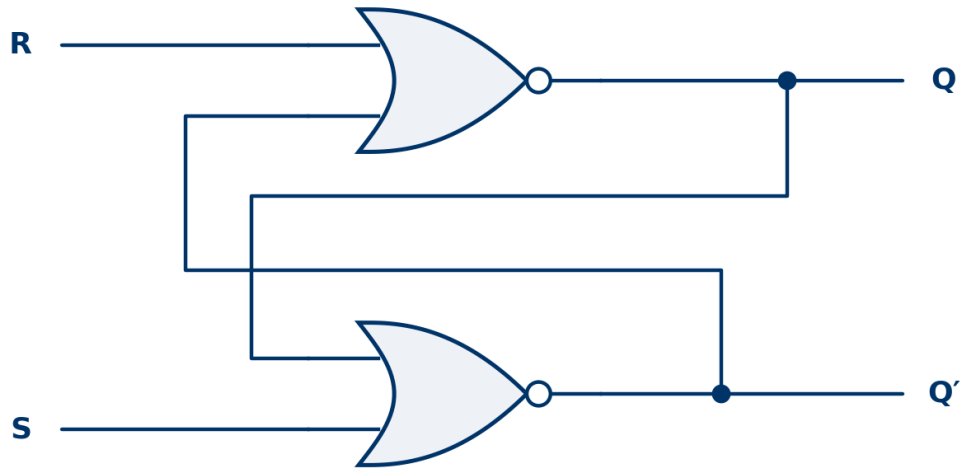
S — **State**
The bits stored in the flip-flops: the circuit's memory.

CK — **Clock**
Periodic pulse that tells every flip-flop when to update.

FB — **Feedback**
Present state returns to the logic and helps decide the next state.

Sequential circuit = combinational logic + memory + feedback, updated by a clock.

The SR Latch: One Bit of Memory



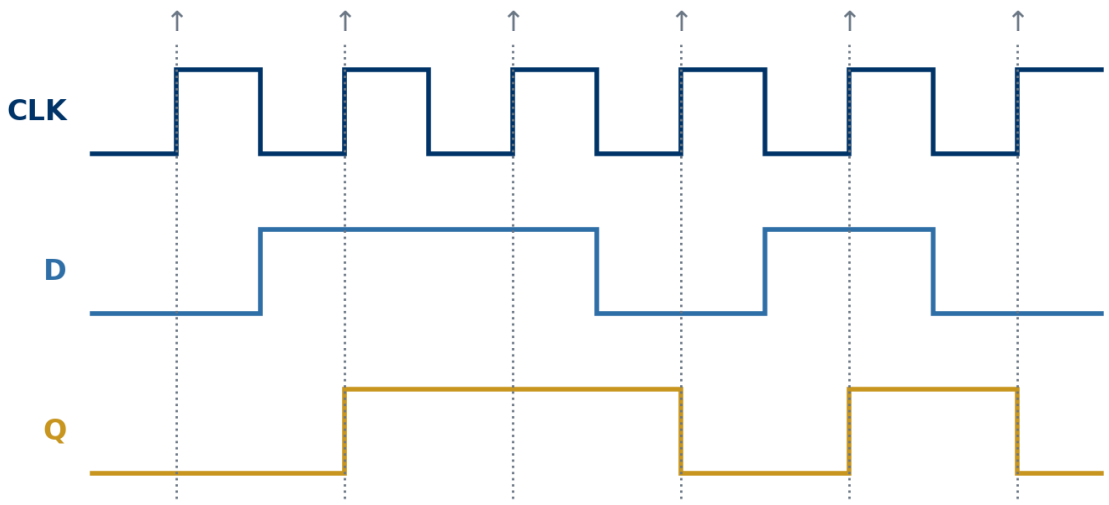
S	R	Q (next)
0	0	No change (memory)
0	1	0 (reset)
1	0	1 (set)
1	1	Invalid: avoid

Two cross-coupled **NOR** gates: each output feeds the other gate.

NAND version uses active-low inputs (S', R').

Cross-coupled feedback gives two stable states: that is a stored bit.

D Latch and D Flip-Flop

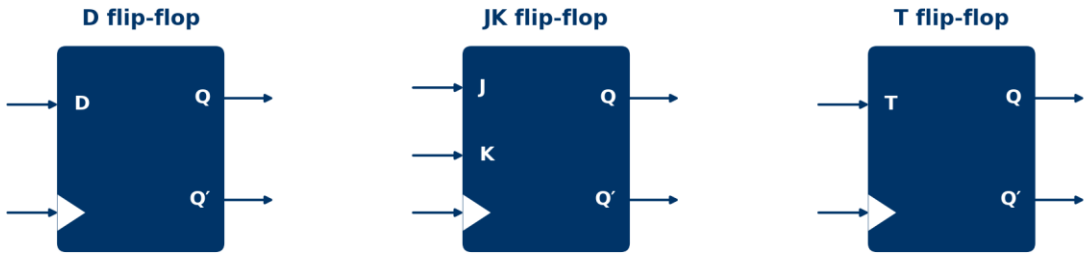


- **D latch:** while the clock is 1 the output follows D (level-sensitive, “transparent”).
- **D flip-flop:** copies D only at the **rising clock edge** (edge-triggered) and holds it until the next edge.
- Built from a **master-slave** pair of D latches with opposite clocks.
- Characteristic equation: **Q(t+1) = D**

D	Q (next)
0	0
1	1

Edge-triggered: Q can change only at the clock edge, which makes synchronous design reliable.

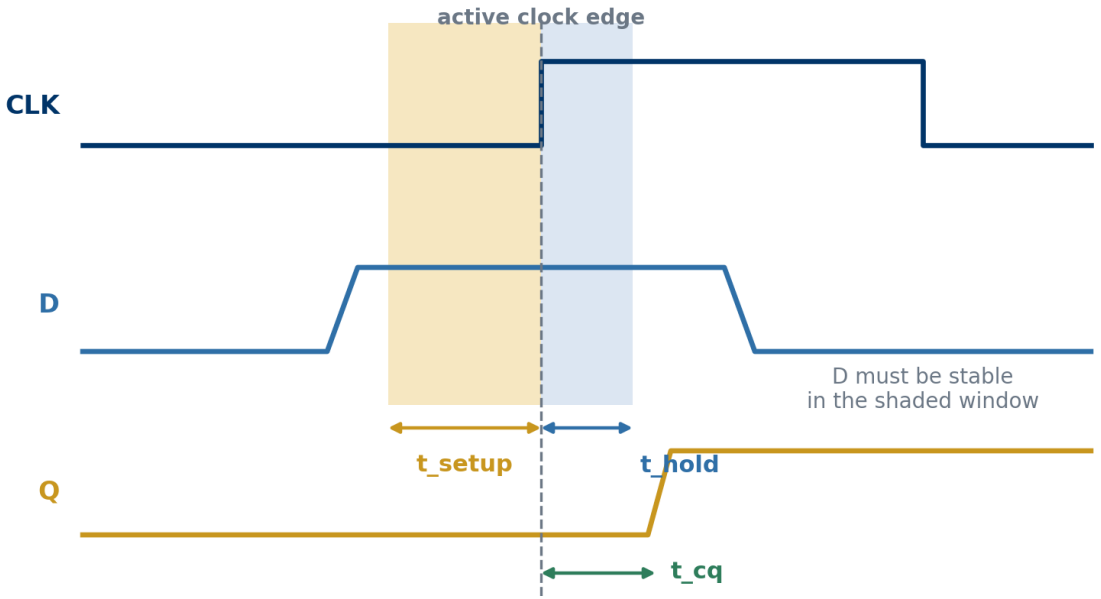
Flip-Flop Types: SR, D, JK and T



J	K	Q (next)
0	0	Q (hold)
0	1	0
1	0	1
1	1	Q' (toggle)

Type	Inputs	Characteristic equation	Behaviour
SR	S, R	Q(t+1) = S + R'Q (SR = 0)	Set / reset / hold; S = R = 1 not allowed
D	D	Q(t+1) = D	Copies D at the clock edge
JK	J, K	Q(t+1) = JQ' + K'Q	Like SR, but J = K = 1 toggles
T	T	Q(t+1) = T ⊕ Q	T = 1 toggles, T = 0 holds

Flip-Flop Timing: Setup, Hold, Delay



If D changes inside the shaded window, the flip-flop may capture a wrong or unstable value (metastability).

t_setup
D stable before edge

t_hold
D stable after edge

t_cq
clock to Q delay

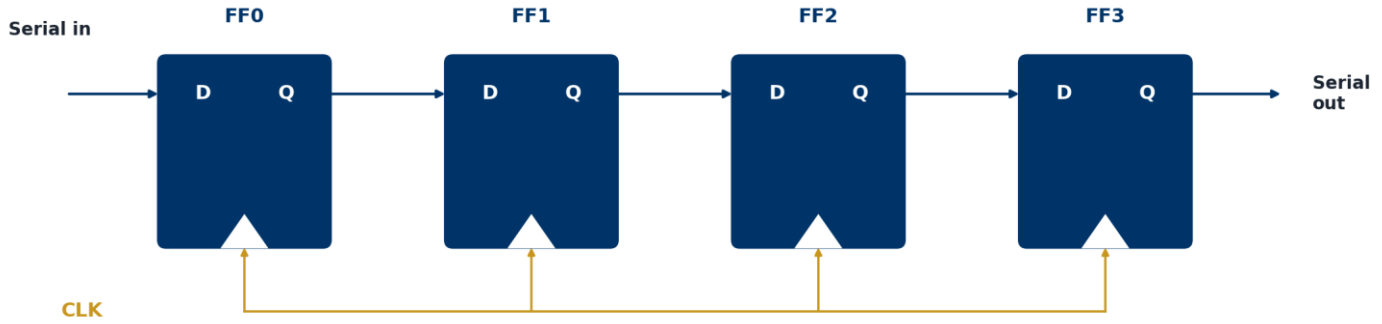
Maximum clock speed

T ≥ t_cq + t_logic + t_setup

0.5 + 3.0 + 0.5 = 4 ns

f_max = 1 / 4 ns = **250 MHz**

Registers and Shift Registers



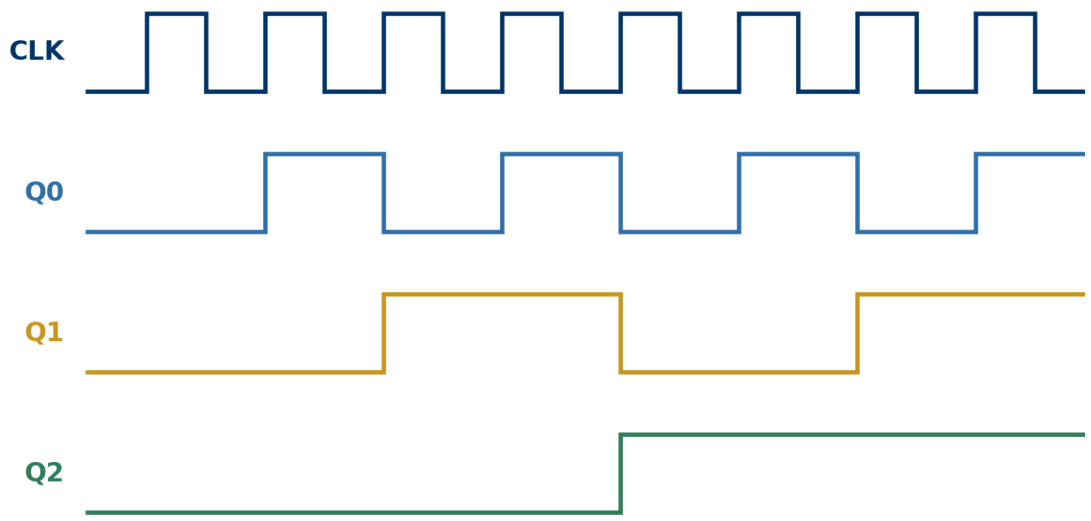
- **Register:** n flip-flops sharing one clock store an n-bit word.
- **Load control:** a MUX or enable decides hold or load.
- **Shift:** each bit moves one place per clock.

Clock	FF0	FF1	FF2	FF3
0 (start)	0	0	0	0
1 (in = 1)	1	0	0	0
2 (in = 0)	0	1	0	0
3 (in = 1)	1	0	1	0
4 (in = 1)	1	1	0	1

Why shifting matters

- Serial ↔ parallel conversion
- Shift left = multiply by 2
- Shift right = divide by 2

Counters: Registers That Count



Q0	Q1	Q2
f / 2	f / 4	f / 8

- **Binary counter:** n flip-flops count 0 to $2^n - 1$, then wrap.
- **Ripple (asynchronous):** each stage is clocked by the previous one: simple, but delay adds up.
- **Synchronous:** all stages share one clock; logic sets each T input: faster.
- **Mod-N counter:** resets after N counts (e.g. mod-10 for decimal).

Count order (Q2 Q1 Q0): 000, 001, …, 111, then 000

Practice Set: Work It Out

#	Question	Hint	Your answer
1	Write S and C equations for a half adder	XOR and AND	
2	Add 1011 + 0110 on a 4-bit ripple adder. List C1 to C4	Add column by column	
3	D flip-flop, D = 1, 0, 1 at three rising edges. Q after each?	Q copies D at edge	
4	JK flip-flop, Q = 0, J = K = 1. Q after 3 clocks?	Toggle each clock	
5	Flip-flops needed for a counter with states 0 to 11?	$2^n \geq 12$	
6	Implement F = A $\oplus$ B with a 4:1 MUX	Read the truth table	

Challenge: a 4-bit shift register starts at 0000. Serial input 1, 1, 0, 1 enters over four clocks. What is stored in FF3 FF2 FF1 FF0?

Real Hardware and Common Mistakes

In real hardware

ALU	Adders and subtractors inside every CPU
MUX	Selecting data sources and ALU operands
REG	CPU registers are rows of D flip-flops
CNT	Program counters, timers, clock dividers

Common mistakes

Latch = flip-flop
Latch follows D while CLK = 1; flip-flop samples only at the edge

S = R = 1 on a NOR latch
Keep it off; use D or JK instead

C0 forgotten
C0 = 0 for add, C0 = 1 for subtract

“Output depends on history” treated as combinational
Needs memory: it is sequential

Every processor repeats the same pattern: adders and muxes compute, flip-flops remember.

Summary and Next Lecture

- **Combinational** circuits have no memory: outputs depend only on present inputs.
- Half adder = XOR + AND; **full adder** = 2 half adders + OR; chain them for n-bit **ripple** addition.
- **Decoders** and **multiplexers** select and route; MUXes implement functions.
- **Sequential** circuits add memory and a clock; **flip-flops** are edge-triggered.
- **Registers** and **counters** are groups of flip-flops; timing limits the clock speed.

Questions?

Next lecture
CPU organisation (control unit, ALU, registers, instruction cycle) and OS structures.

Lab
Simulate half and full adders, a ripple adder, latches, flip-flops and counters.

Reading
Mano: Combinational Logic, Sequential Logic, Registers and Counters. H&P Ch. 1.